

VarBlockSpMM: Supplementary Evidence

Cohen Brunner Thomas

6 September 2026

1 Original NVIDIA controls

This table and figure use the original seed-1, three-process campaign. The comparator set includes cuSPARSE and grouped cuBLAS, without MAGMA. They remain separate from the focused MAGMA follow-up in the main paper.

Table 1: Original width breakdown, with process-averaged ratios.

RHS	CSR	Grouped	BSR8	Best library	Direct wins	Min
8	3.513	9.116	1.078	1.057	26/32	0.951
16	4.415	7.778	1.086	1.042	24/32	0.823
32	4.100	4.451	1.128	1.087	21/32	0.903
64	5.190	3.481	1.444	1.397	32/32	1.112

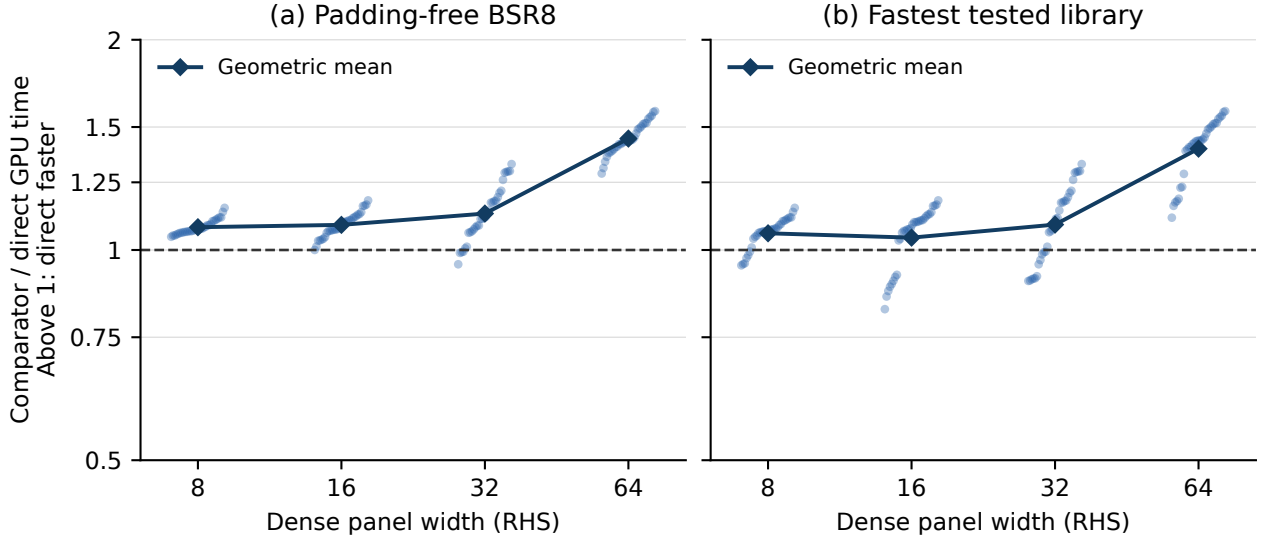


Figure 1: Original core width comparison. Each point averages three process ratios. The fastest-library panel uses the original NVIDIA controls.

Table 2: Descriptive process-label ranges on the core grid. The two endpoints are observed aggregate ratios, not confidence limits.

Comparator	Geometric mean	Process-label range
CSR default	6.834	6.830 to 6.838
CSR lower envelope	4.262	4.253 to 4.268
Grouped cached	5.757	5.755 to 5.759
Grouped changing	5.752	5.749 to 5.754
Subdivided BSR8	1.175	1.174 to 1.176

Table 3: Width and degree breakdown for the core grid. Times are milliseconds and speedups are dimensionless.

RHS	Degree	Direct ms	CSR ratio	Grouped ratio	BSR8 ratio
8	2	0.114	2.643	7.839	1.105
8	4	0.207	3.667	8.901	1.081
8	8	0.386	5.598	9.741	1.068
8	16	0.751	2.807	10.158	1.058
16	2	0.145	3.553	6.244	1.052
16	4	0.248	4.140	7.577	1.069
16	8	0.447	5.643	8.647	1.109
16	16	0.873	4.576	8.945	1.117
32	2	0.283	3.456	3.461	1.086
32	4	0.508	3.655	4.073	1.039
32	8	0.836	4.476	5.083	1.177
32	16	1.582	4.998	5.480	1.218
64	2	0.415	4.619	2.731	1.445
64	4	0.727	4.995	3.416	1.434
64	8	1.352	5.278	3.825	1.436
64	16	2.588	5.960	4.116	1.461

Table 4: Size and degree-pattern controls with random column locality. Each row averages eight configurations and three process observations.

Block rows	Degree pattern	CSR ratio	Grouped ratio	CSR wins
256	16 in every row	2.870	6.428	8/8
256	0, 1, 16, 16	1.492	4.826	8/8
1024	16 in every row	3.056	5.552	8/8
1024	0, 1, 16, 16	3.677	5.294	8/8
4096	16 in every row	4.082	6.224	8/8
4096	0, 1, 16, 16	4.763	5.629	8/8

Table 5: Matched control time divided by changed-policy time. RHS-8 effects are geometric marginal contrasts over the other two factorial choices. Minima and maxima first average three process ratios for each fixed matrix instance. They are observed ranges, not confidence limits.

RHS	Change	Ratio	Min	Max
8	Eight-row mapping	1.075	0.995	1.153
8	Two accumulators	0.972	0.825	1.084
8	One CTA instead of two	0.943	0.837	1.011
8	Full-panel staging	1.472	1.120	1.966
16	Wider accumulation	1.084	0.918	1.506
16	Cooperative staging	1.406	1.102	1.897
16	Double buffering	1.022	0.925	1.104
16	Full-panel staging	2.271	1.866	2.997
32	Wider accumulation	1.310	1.113	1.598
32	Cooperative staging	1.414	1.205	1.770
32	Double buffering	1.012	0.846	1.191
64	Wider accumulation	1.379	1.217	1.731
64	Cooperative staging	1.632	1.493	1.857
64	Double buffering	0.801	0.661	0.924

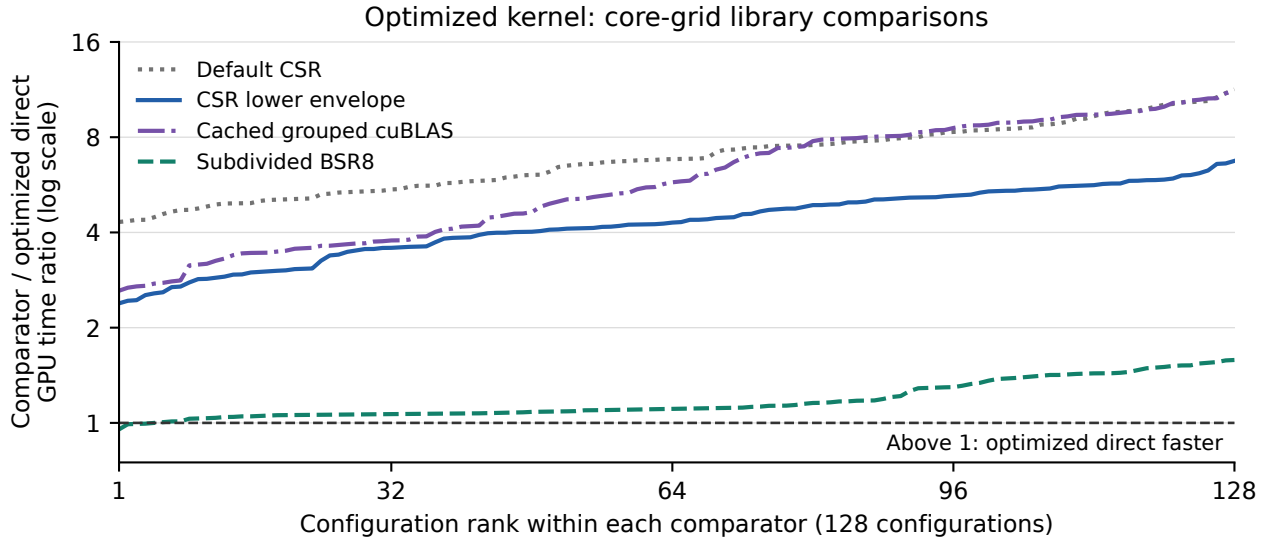


Figure 2: GPU time ratios across the 128 core configurations. Each ratio is the geometric mean of three paired process ratios, computed from per-process timing medians. Each curve sorts configurations independently, so equal ranks need not identify the same matrix. The vertical axis is logarithmic. Ratios above the dashed parity line favor direct execution. These are empirical distributions over the chosen grid, not uncertainty intervals.

Table 6: All published-matrix controls, averaged across twelve matrix and width configurations. Ratios divide comparator GPU time by direct time.

Comparator	Ratio	Minimum	Direct wins
Compact CSR	0.352	0.210	0/12
BSR8	0.648	0.498	0/12
Grouped cached	5.246	3.361	12/12
Dense SGEMM	1.051	0.460	7/12

Table 7: Near-field comparator GPU time divided by direct time. Each row averages three geometry seeds and three processes per seed. Ratios above one favor direct execution.

Points	Geometry	RHS	CSR	BSR8	Grouped	Dense
2048	uniform	8	0.466	0.371	19.250	0.136
2048	uniform	16	0.557	0.276	13.892	0.121
2048	uniform	32	1.070	0.300	14.079	0.166
2048	uniform	64	0.871	0.252	9.183	0.124
2048	clustered	8	1.012	0.502	16.778	0.163
2048	clustered	16	1.188	0.382	12.269	0.165
2048	clustered	32	1.581	0.286	8.679	0.150
2048	clustered	64	1.754	0.425	7.417	0.129
8192	uniform	8	0.535	0.372	22.483	1.185
8192	uniform	16	0.620	0.253	13.837	1.412
8192	uniform	32	1.032	0.325	11.958	0.490
8192	uniform	64	1.079	0.304	6.529	0.263
8192	clustered	8	1.181	0.598	13.678	0.816
8192	clustered	16	1.836	0.513	10.906	1.256
8192	clustered	32	2.350	0.668	7.097	0.331
8192	clustered	64	3.190	0.836	4.947	0.224

1.1 Core grid and format controls

The original seed-1 campaign below uses three processes per configuration and predates the MAGMA comparison. Its aggregates remain separate. Table 8 compares execution times on the core grid. The direct path has a geometric speedup of 4.262 against the CSR lower envelope and 5.757 against cached grouped cuBLAS. The respective win counts are 128 and 128 out of 128 configurations. Selecting the CSR algorithm

reduces the aggregate ratio from 6.834 for default CSR to 4.262 for the lower envelope. The subdivided BSR8 ratio is 1.175, with 124 direct wins out of 128. This comparison retains block structure without padding, comparing two ways to execute the same dense blocks. Figure 2 shows the spread behind these averages. Its sorted curves expose the fastest comparators and the width of the performance range across the grid.

Table 8: Core-grid ratios over 128 configurations and three process observations per configuration. Minima and wins use GPU ratios averaged in log space across processes. Changing-address grouped execution is paired with changing-address direct execution.

Comparator	GPU ratio	Host ratio	Minimum	Wins
CSR default	6.834	6.732	4.317	128/128
CSR lower envelope	4.262	4.209	2.387	128/128
Grouped cached	5.757	5.668	2.617	128/128
Grouped changing	5.752	5.661	2.626	128/128
Subdivided BSR8	1.175	1.172	0.954	124/128

Table 3 breaks down the same results by width and degree. Each row averages the four size distributions and two localities. Direct time is the geometric mean of the corresponding process medians. Absolute times distinguish small launch-sensitive cases from larger products. The CSR column uses the lower envelope, and grouped execution uses cached pointers.

1.2 Additional seeds and queued comparisons

The follow-up repeats all 128 core configurations on matrix seeds 2, 3, and 5. Each seed and timing mode uses one fresh process per configuration, giving 768 processes. The two modes synchronize after either one product or eight queued products. Each process measures twenty samples, with GPU and host times divided by the number of products in the sample. The eight products use the same prepared plan and buffers. Changing-address controls alternate the two buffer pairs on each product. Validation runs outside timing, before and after the samples, and outputs are poisoned before each method. Jobs and methods are randomized, and GPU processes run serially.

Table 9 uses the same library lower envelope as the original core comparison. Each row includes 32 configurations on each of three matrix seeds. The seed range contains the smallest and largest 32-configuration geometric means. These ranges describe matrix variation combined with process noise. There is one process for each seed and mode, so they do not separate those sources or define confidence intervals. The original three-process campaign remains separate. The follow-up uses C++20 host compilation and the same library source bytes and CUDA toolkit.

Table 9: Additional matrix seeds and synchronization intervals for the NVIDIA controls. Ratios divide comparator time by direct time, so values above one favor direct execution. Batch is the number of products queued before synchronization. Wins compare direct execution with the fastest tested library for each of 96 instances.

Batch	RHS	Best library	BSR8	Wins	Seed range
1	8	1.056	1.081	75/96	1.052 to 1.060
1	16	1.046	1.091	72/96	1.045 to 1.047
1	32	1.091	1.131	65/96	1.087 to 1.094
1	64	1.395	1.443	96/96	1.393 to 1.399
8	8	1.050	1.074	73/96	1.049 to 1.052
8	16	1.034	1.079	72/96	1.032 to 1.036
8	32	1.086	1.125	64/96	1.075 to 1.104
8	64	1.392	1.452	95/96	1.384 to 1.398

Across all 384 instances per mode, the fastest-NVIDIA-library ratios are 1.139 for individual products and 1.132 for batches of eight, with 308 and 304 direct wins respectively. Every width has a geometric-mean advantage on each additional seed in both timing modes. RHS 64 gives the largest aggregate advantage, about 1.39, winning all 96 individual-product instances and 95 of 96 queued instances. The narrow-panel gains remain smaller, about three to six percent against the fastest tested library. Queueing eight products preserves this width-dependent pattern on the measured grid.

2 Transport discretization and follow-up controls

For element width h , the modal mass matrix has entries $M_{ii} = h/(2i + 1)$. Set the characteristic displacement fraction to $\theta = \Delta t/h = 1/4$. Exact translation followed by elementwise L^2 projection gives the stored one-step blocks

$$A_{ij}^{\text{self}} = \frac{2i+1}{2} \int_{-1+2\theta}^1 P_i(\xi) P_j(\xi - 2\theta) d\xi,$$

$$A_{ij}^{\text{upwind}} = \frac{2i+1}{2} \int_{-1}^{-1+2\theta} P_i(\xi) P_j(\xi + 2 - 2\theta) d\xi.$$

The 96-point Gauss-Legendre rule integrates these polynomial products exactly up to floating-point roundoff. Element orders repeat 7, 15, 31, 63. The block product retains each element's own basis and its upstream basis, without scalar reblocking. This is a semi-Lagrangian DG update, replacing the original forward-Euler experiment. Orthogonal projection after translation is nonexpansive in physical L^2 in exact arithmetic.

Initialization projects $\sin(2\pi kx)$ for $k = 1, \dots, N$. All performance cases reach $T = 1/128$. Their 256, 1,024, and 4,096 elements require 8, 32, and 128 products, respectively. Analytic comparisons project $\sin(2\pi k(x - T))$ and weight coefficient errors by $h/(2i + 1)$. Every measured method checks its actual GPU result against that physical reference as well as the independent repeated sparse-product reference. An unchanged-state operation must fail the coefficient tolerances before any timing starts. All six prepared inputs exceed a tenfold rejection margin in both maximum absolute and relative coefficient error.

A separate FP64 spatial-refinement check keeps $T = 1/16$, the eight initial waves, the repeating order schedule, and $\theta = 1/4$ fixed. With 16, 32, and 64 elements, the analytic relative errors are 1.059×10^{-6} , 6.253×10^{-9} , and 3.478×10^{-11} . The preparation script rejects a failure to reduce error by at least a factor of four at each refinement. This is a smooth-wave spatial-refinement check, not evidence for arbitrary nonlinear problems. The FP64 check isolates discretization behavior from quantized FP32 arithmetic.

The trace comparator starts from identical prepared host blocks. Startup includes method construction and one complete trace, including the reset of its initial device buffer and output poisoning. Common panel allocations and Python assembly are excluded. Every method's output is checked against repeated FP64 sparse products using the assembled FP32 coefficients. No CPU reference work occurs inside timing.

MAGMA slot execution needs no product workspace. The reduction composition allocates $64N$ FP32 entries per stored block. The alternating-buffer transport trace retains two pointer plans and thus twice that product workspace. The reduction kernel and all repeated slot calls are timed. All MAGMA code is pinned in the source archive. This composition comparison does not reproduce the full hierarchical solver from the motivating work.

Table 10: Controlled combinations. Best-library-to-direct ratios average three paired process ratios. Fixed-size 32 cases include BSR32.

Rows	Degree	Size	RHS	Direct ms	BSR8	MAGMA best	Best library
128	4	8	8	0.012	0.847	1.154	0.785
128	4	32	8	0.017	0.869	1.507	0.869
128	4	8	64	0.019	0.852	1.238	0.799
128	4	32	64	0.027	1.111	1.920	0.881
128	16	8	8	0.025	0.537	1.152	0.537
128	16	32	8	0.041	0.820	1.448	0.820
128	16	8	64	0.033	0.458	1.919	0.458
128	16	32	64	0.074	1.295	1.700	0.984
128	128	8	8	0.138	0.372	1.039	0.372
128	128	32	8	0.501	0.661	0.729	0.661
128	128	8	64	0.190	0.317	2.409	0.317
128	128	32	64	0.631	1.108	1.807	0.817
512	4	8	8	0.013	0.680	1.971	0.680
512	4	32	8	0.021	1.687	2.505	1.312
512	4	8	64	0.035	0.550	1.682	0.550
512	4	32	64	0.066	1.408	1.924	0.975
512	16	8	8	0.031	0.461	2.405	0.461
512	16	32	8	0.076	1.266	2.465	0.937
512	16	8	64	0.109	0.278	2.061	0.278
512	16	32	64	0.316	1.203	1.670	0.849
512	128	8	8	0.193	0.300	2.735	0.300
512	128	32	8	0.679	1.053	2.113	1.028
512	128	8	64	0.788	0.226	2.363	0.226
512	128	32	64	2.390	1.405	1.824	1.108

Update paths. On the 256-block-row RHS-32 API test, value replacement plus execution takes 0.058 ms in synchronized host time, versus 0.213 ms for borrowing changed structure, validating its metadata, rebuilding the row order, and executing. These are geometric means of three process medians, with twenty samples each. Values alternate between two pre-existing device payloads on fixed degree-four structure. Structural cases alternate between degree-four and degree-eight matrices already on the GPU. Thus this is an API-path cost comparison, not an equal-work kernel speedup or a measurement of assembling a new structure.

3 Numerical behavior and input census

Device sources are compiled with `--use_fast_math`, which implies `-ftz=true` (see the CUDA compiler reference in the main paper). Each output entry is a sum of products evaluated as FP32 multiply-add with FP32 accumulation, in an unspecified order, with no compensated summation. Flush-to-zero applies per instruction, to that instruction’s inputs and to its rounded result. It does not apply to the product term inside a fused multiply-add, which is computed exactly and added to the accumulator before a single rounding, so `fma(FLT_MIN, 0.5, FLT_MIN)` retains `1.5 FLT_MIN`. The consequences still cannot be excluded by inspecting the caller’s inputs. An instruction input, or an accumulator value an instruction produces, that lands in the subnormal range is replaced by zero, so contributions that pass through a subnormal accumulator are lost: two contributions of $\text{FLT_MIN} \times 0.5$ have normal operands and an exact sum of `FLT_MIN`, and the kernel returns zero at every supported width, because each multiply-add produces a subnormal result from a zero accumulator. A partial sum above `FLT_MAX` becomes an infinity even when the exact sum is finite, and the unspecified order means a later cancelling term cannot be relied on. Accumulated error otherwise grows with the number of terms and with cancellation, as in any uncompensated FP32 summation.

The claim is therefore verified rather than guaranteed. On the tested workloads, results agree with a reference that accumulates in double and rounds once, with maximum absolute error at most 5×10^{-4} and relative L2 error at most 5×10^{-5} . A dedicated range test pins each behavior above, including the multi-contribution cases.

The measured inputs are not uniformly in the normal range. The three clustered 2,048-point covariance matrices contain 162, 212, and 294 subnormal packed values. Those values carry less than 10^{-30} of each matrix’s total magnitude, so their numerical contribution is negligible, and the measured outputs pass the stated correctness checks. That is a statement about numerical contribution, not about time: we did not

measure these workloads with and without flush-to-zero, so nothing here establishes that the timing ratios are unaffected. A checked-in census keeps these counts tied to the payloads. It covers the covariance and published matrices and all six transport configurations, including the transport initial states, references, and mass weights, which are operands as well. Those contain no subnormal values.